Sliding Window Median (hard)

We'll cover the following

Problem Statement

Given an array of numbers and a number ‘k’, find the median of all the ‘k’ sized sub-arrays (or windows) of the array.

Example 1:

Input: nums=[1, 2, -1, 3, 5], k = 2
Output: [1.5, 0.5, 1.0, 4.0]
Explanation: Lets consider all windows of size ‘2’:

- [**1, 2**, -1, 3, 5] -> median is 1.5
- [1, **2, -1**, 3, 5] -> median is 0.5
- [1, 2, **-1, 3**, 5] -> median is 1.0
- [1, 2, -1, **3, 5**] -> median is 4.0

Example 2:

Input: nums=[1, 2, -1, 3, 5], k = 3
Output: [1.0, 2.0, 3.0]
Explanation: Lets consider all windows of size ‘3’:

- [**1, 2, -1**, 3, 5] -> median is 1.0
- [1, **2, -1, 3**, 5] -> median is 2.0
- [1, 2, **-1, 3, 5**] -> median is 3.0

Try it yourself

Try solving this question here:

Java | Python3 | JS | C++

```java
import java.util.*;

class SlidingWindowMedian {
  public double[] findSlidingWindowMedian(int[] nums, int k) {
    double[] result = new double[nums.length - k + 1];
    // TODO: Write your code here
    return result;
  }

  public static void main(String[] args) {
    SlidingWindowMedian slidingWindowMedian = new SlidingWindowMedian();
    double[] result = slidingWindowMedian.findSlidingWindowMedian(new int[] { 1, 2, -1, 3, 5 }, 2);
    System.out.print("Sliding window medians are: ");
    for (double num : result)
      System.out.print(num + " ");
    System.out.println();

    slidingWindowMedian = new SlidingWindowMedian();
    result = slidingWindowMedian.findSlidingWindowMedian(new int[] { 1, 2, -1, 3, 5 }, 3);
    System.out.print("Sliding window medians are: ");
    for (double num : result)
      System.out.print(num + " ");
  }

}
```

Run Save Reset

Solution

This problem follows the **Two Heaps** pattern and share similarities with Find the Median of a Number Stream. We can follow a similar approach of maintaining a max-heap and a min-heap for the list of numbers to find their median.

The only difference is that we need to keep track of a sliding window of 'k' numbers. This means, in each iteration, when we insert a new number in the heaps, we need to remove one number from the heaps which is going out of the sliding window. After the removal, we need to rebalance the heaps in the same way that we did while inserting.

Code

Here is what our algorithm will look like:

Java Python3 C++ JS

```java
import java.util.*;

class SlidingWindowMedian {
  PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
  PriorityQueue<Integer> minHeap = new PriorityQueue<>();

  public double[] findSlidingWindowMedian(int[] nums, int k) {
    double[] result = new double[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
      if (maxHeap.size() == 0 || maxHeap.peek() >= nums[i]) {
        maxHeap.add(nums[i]);
      } else {
        minHeap.add(nums[i]);
      }
      rebalanceHeaps();

      if (i - k + 1 >= 0) { // if we have at least 'k' elements in the sliding window
```

```
      if (i - k + 1 >= 0) { // if we have at least 'k' elements in the sliding window
        // add the median to the the result array
        if (maxHeap.size() == minHeap.size()) {
          // we have even number of elements, take the average of middle two elements
          result[i - k + 1] = maxHeap.peek() / 2.0 + minHeap.peek() / 2.0;
        } else { // because max-heap will have one more element than the min-heap
          result[i - k + 1] = maxHeap.peek();
        }

        // remove the element going out of the sliding window
        int elementToBeRemoved = nums[i - k + 1];
        if (elementToBeRemoved <= maxHeap.peek()) {
          maxHeap.remove(elementToBeRemoved);
        } else {
          minHeap.remove(elementToBeRemoved);
        }
        rebalanceHeaps();
      }
    }
    return result;
  }

  private void rebalanceHeaps() {
    // either both the heaps will have equal number of elements or max-heap will have
    // one more element than the min-heap
    if (maxHeap.size() > minHeap.size() + 1)
      minHeap.add(maxHeap.poll());
    else if (maxHeap.size() < minHeap.size())
      maxHeap.add(minHeap.poll());
  }

  public static void main(String[] args) {
    SlidingWindowMedian slidingWindowMedian = new SlidingWindowMedian();
    double[] result = slidingWindowMedian.findSlidingWindowMedian(new int[] { 1, 2, -1, 3, 5 }, 2);
    System.out.print("Sliding window medians are: ");
    for (double num : result)
      System.out.print(num + " ");
    System.out.println();

    slidingWindowMedian = new SlidingWindowMedian();
    result = slidingWindowMedian.findSlidingWindowMedian(new int[] { 1, 2, -1, 3, 5 }, 3);
    System.out.print("Sliding window medians are: ");
    for (double num : result)
      System.out.print(num + " ");
  }

}
```

Run Save Reset

Time complexity

The time complexity of our algorithm is $O(N * K)$ where 'N' is the total number of elements in the input array and 'K' is the size of the sliding window. This is due to the fact that we are going through all the 'N' numbers and, while doing so, we are doing two things:

1. Inserting/removing numbers from heaps of size 'K'. This will take $O(logK)$
2. Removing the element going out of the sliding window. This will take $O(K)$ as we will be searching this element in an array of size 'K' (i.e., a heap).

Space complexity

Ignoring the space needed for the output array, the space complexity will be $O(K)$ because, at any time, we will be storing all the numbers within the sliding window.

Back

Mark As Completed Next

Find the Median of a Number Stream (medium)

Maximize Capital (hard)